

Netplix

Rapport du projet

Mohamed Yassine Agourram

Kun Li

Sorbonne Université - Faculté des Sciences et Ingénierie

LO3IN403 - Projet OPsci

Avril 2026



1 Introduction	4
1.1 Présentation principale	4
1.2 Graphe pédagogique	5
1.3 Périmètre fonctionnel	5
2 Partie théorique	7
2.1 Étude des architectures logicielles existantes	7
2.2 Étude des Design Patterns	8
2.3 Étude comparative des technologies Back-end	10
2.4 Étude comparative des technologies Front-end	11
2.5 Concepts fondamentaux du Web et de l'API	11
2.6 Communication front-end / back-end	13
2.7 Bonnes pratiques avancées (API réelle)	14
2.8 Automatisation : Théorie du CI/CD	15
2.9 Docker	15
2.10 Orchestration de conteneurs avec Kubernetes	16
3 Partie Pratique	18
3.1 Contexte général - Netplix	18
3.1.1 Idée du projet	18
3.1.2 Inspiration	18
3.1.3 Architecture choisie	19
3.1.4 Design patterns choisie	19
3.1.5 Explication de la stack technologique choisie	20
3.1.6 Organisation du travail	21
3.2 Source des données	21
3.2.1 Choix du dataset	21
3.2.1 Import et normalisation des données	23
3.2.3 Import automatique au démarrage	24
3.3 Back-end : FastAPI et MongoDB	24
3.3.2 API principale - main.py	25

3.3.3 Elasticsearch - Recherche avancée	27
3.4 Front-end HTML/CSS/JavaScript	28
3.4.1 Structure HTML	28
3.4.2 Design CSS	29
3.4.3 Logique JavaScript	29
3.5 Communication Front-end/Back-end	30
3.5.1 Le protocole REST	30
3.5.2 Format de réponse uniforme	31
3.5.3 Gestion des erreurs HTTP	32
3.6 Conteneurisation avec Docker	32
3.6.1 Bonne pratique : .dockerignore	32
3.6.2 Dockerfile du back-end	32
3.6.3 Dockerfile du front-end	33
3.6.4 Docker compose	33
3.6.5 Réseau Docker	33
3.6.6 Problèmes rencontrée	33
3.7 Orchestration avec Kubernetes	34
3.7.1 Organisation en Namespace	34
3.7.2 ConfigMap - Configuration centralisée	35
3.7.3 Deployment du back-end	35
3.7.4 Deployment du front-end	35
3.7.5 L'Ingress - Routeur HTTP	35
3.7.6 Services Redis et Elasticsearch déployés	36
3.8 CI/CD et dépôt Git	36
3.8.1 Déroulement du pipeline	36
4 Conclusion	37
5 Références	39

1 Introduction

1.1 Présentation principale

Ce projet s'inscrit dans le cadre du module LU3IN403 de la Licence 3 Informatique à Sorbonne Université. L'objectif est de concevoir, implémenter et déployer une application web complète, proche des pratiques professionnelles actuelles.

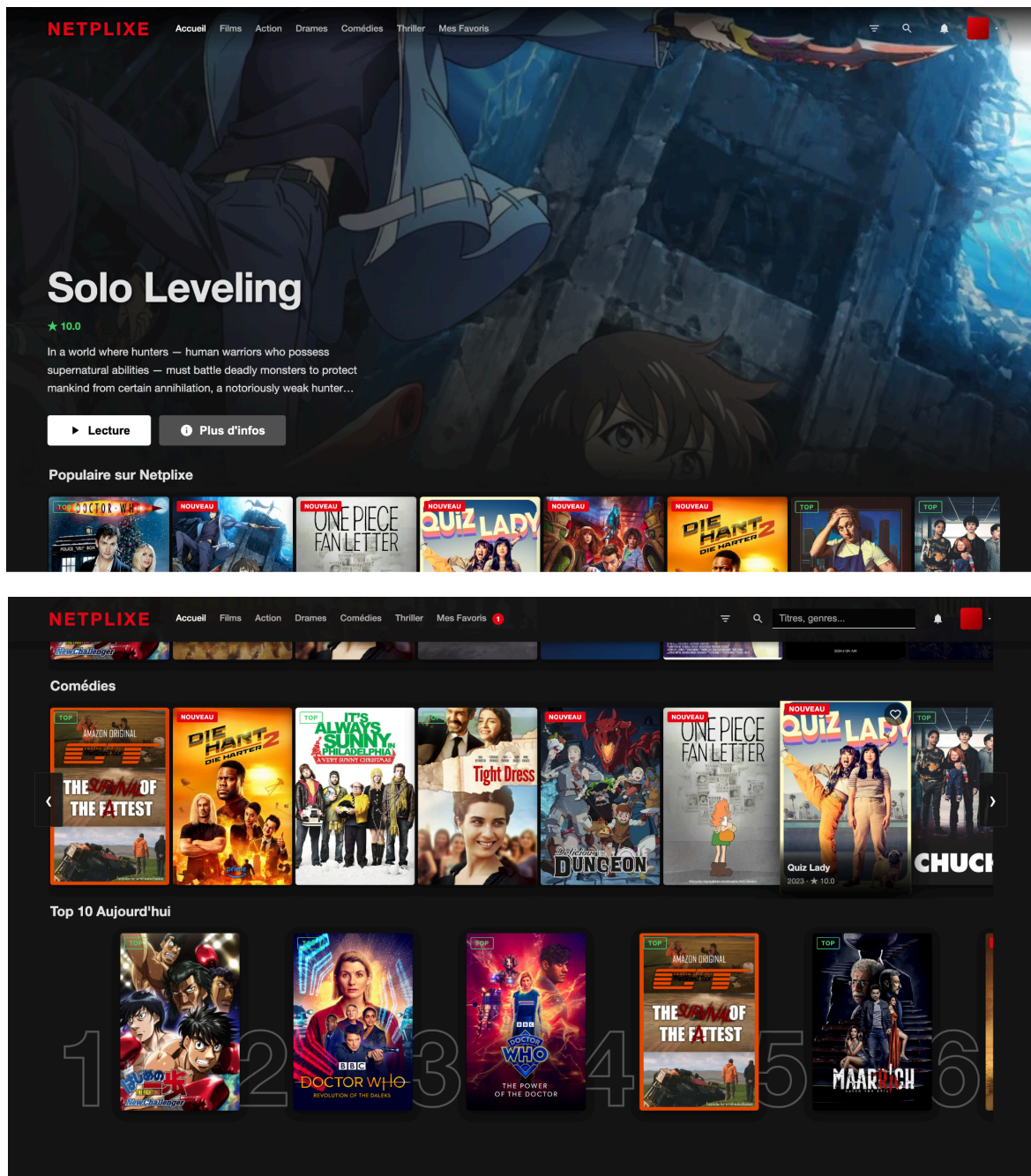


Figure : Page principale du Netplixe

L'application réalisée est **Netplix**, un catalogue de films inspiré de l'interface de Netflix. Elle permet aux utilisateurs de parcourir un catalogue de plus de 25 000 films, de les rechercher, de les filtrer par genre ou par note, de consulter une fiche détaillée pour chaque film, de sauvegarder leurs favoris et d'obtenir des recommandations de films similaires.

1.2 Graphe pédagogique

Le projet vise à démontrer la maîtrise des compétences suivantes :

- Conception d'une architecture web moderne découplée en couches (front-end / back-end / base de données)
- Développement d'une API RESTful avec FastAPI (Python)
- Utilisation d'une base de données NoSQL (MongoDB)
- Implémentation de fonctionnalités avancées : cache Redis, moteur de recherche Elasticsearch, pagination
- Conteneurisation de l'ensemble de l'application avec Docker
- Orchestration des conteneurs avec Kubernetes (Minikube)
- Mise en place d'un pipeline CI/CD automatisé avec GitHub Actions
- Justification des choix techniques dans un rapport structuré

1.3 Périmètre fonctionnel

Fonctionnalités de base :

- Affichage d'un catalogue dynamique de films organisé en rangées horizontales (Netflix-style)
- Recherche par titre et description (insensible à la casse)
- Filtrage par type (Film), par genre (Action, Drame, Comédie, Thriller)
- Fiche détaillée de chaque film avec affiche, description, note et métadonnées

Fonctionnalités avancées :

- Favoris : sauvegarde des films préférés avec persistance localStorage
- Films similaires : recommandations basées sur les genres partagés
- Cache Redis : mise en cache de toutes les réponses API (TTL configurable)
- Recherche avancée Elasticsearch : recherche multi-critères avec tolérance aux fautes
- Pagination : navigation par pages avec compteurs et boutons dynamiques

2 Partie théorique

2.1 Étude des architectures logicielles existantes

L'Architecture Monolithique

Dans un monolithe, l'ensemble de l'application (UI, logique métier et accès aux données) est développé, compilé et déployé comme une seule unité logique. Tout le code partage la même mémoire et les mêmes ressources.

- **Avantages :** Simplicité de développement et de déploiement initial.
 - Performances élevées (pas d'appels réseau entre les composants).
 - Facilité de test de bout en bout.
- **Limites :** "Spaghetti code" : avec le temps, les composants deviennent trop entrelacés.
 - Scalabilité difficile : on est obligé de dupliquer toute l'application même si seul un petit module est surchargé.
 - Risque systémique : une erreur critique dans un module peut faire tomber toute l'application.
- **Cas d'usage :** Petites applications, preuves de concept (PoC) ou équipes très réduites.

L'Architecture Client-Serveur

Ce modèle sépare l'application en deux entités : le Client (l'interface utilisateur) qui initie des requêtes, et le Serveur qui traite ces requêtes et renvoie les ressources. Ils communiquent généralement via le protocole HTTP.

- **Avantages :** Séparation des responsabilités (UI vs Logique).
 - Indépendance technologique : on peut changer le langage du client sans toucher au serveur.
 - Centralisation des données et de la sécurité sur le serveur.
- **Limites :** Dépendance totale à la connectivité réseau.
 - Le serveur peut devenir un "goulot d'étranglement" s'il reçoit trop de requêtes.
- **Cas d'usage :** La majorité des sites web modernes et des applications mobiles.

L'Architecture en Couches (Layered Architecture)

Également appelée "n-tier", cette architecture organise le code en strates horizontales (généralement : Présentation, Application, Domaine/Métier, Infrastructure/Données). Chaque couche a un rôle précis et ne peut communiquer qu'avec la couche immédiatement inférieure.

- **Avantages :** Très forte maintenabilité : le code est rangé logiquement.
 - Facilité de remplacement : on peut changer la base de données sans modifier l'interface.
 - Testabilité accrue grâce au découplage.
- **Limites :** "Effet de cascade" : un changement mineur en bas peut parfois remonter toutes les couches.
 - Peut introduire une complexité inutile pour des fonctions très simples.
- **Cas d'usage :** Logiciels d'entreprise complexes et robustes.

L'Architecture Microservices

L'application est décomposée en une suite de services autonomes, spécialisés dans une fonction métier unique (ex: service de recherche, service de notation, service utilisateur). Chaque service possède sa propre base de données.

- **Avantages :** Scalabilité granulaire : on multiplie uniquement les services les plus utilisés.
 - Résilience : la panne d'un microservice ne paralyse pas l'ensemble du système.
 - Agilité : chaque équipe peut travailler sur son propre service avec ses propres technos.
- **Limites :** Complexité opérationnelle extrême (réseau, déploiement, surveillance).
 - Difficultés à maintenir la cohérence des données entre les services.
- **Cas d'usage :** Systèmes à très grande échelle (Netflix, Amazon, Uber).

2.2 Étude des Design Patterns

Un design pattern est une solution générique à un problème récurrent de conception logicielle.

MVC (Model-View-Controller)

- **Problème résolu :** Le mélange du code d'interface utilisateur avec la logique métier et la gestion des données.

- **Présentation** : Ce pattern divise l'application en trois composants : le Modèle (données), la Vue (interface) et le Contrôleur (logique de lien).
- **Avantages** : Permet de modifier l'interface sans toucher à la logique de données. Facilite le travail en équipe.
- **Limites** : Peut devenir complexe à gérer si les interactions entre la vue et le modèle sont trop nombreuses.

Repository

- **Problème résolu** : Le couplage direct entre la logique de l'application et le système de stockage des données (Base de données, fichier JSON).
- **Présentation** : Il agit comme une couche d'abstraction (une interface) entre la logique métier et la source de données.
- **Avantages** : On peut changer de base de données (ex: passer de JSON à PostgreSQL) sans modifier une seule ligne de la logique métier.
- **Limites** : Ajoute une couche de code supplémentaire qui peut sembler inutile pour des applications très simples.

Singleton

- **Problème résolu** : La création multiple et inutile d'objets gourmands en ressources (comme une connexion à une base de données).
- **Présentation** : Garantit qu'une classe ne possède qu'une seule et unique instance accessible globalement.
- **Avantages** : Économise de la mémoire et centralise l'accès à une ressource partagée.
- **Limites** : Introduit un état global, ce qui peut rendre les tests unitaires plus difficiles.

Factory (Fabrique)

- **Problème résolu** : La complexité liée à l'instanciation d'objets sans connaître exactement leur type final au moment de la rédaction du code.
- **Présentation** : Fournit une interface pour créer des objets dans une classe mère, tout en laissant les sous-classes décider du type d'objet à créer.
- **Avantages** : Découple le code qui utilise l'objet du code qui le crée.

- **Limites** : Rend le code plus abstrait et parfois plus difficile à lire pour les débutants.

Observer

- **Problème résolu** : La nécessité de notifier plusieurs composants lorsqu'un changement survient dans un objet, sans que cet objet ne connaisse ses observateurs.
- **Présentation** : Un objet "Sujet" maintient une liste de ses dépendants "Observateurs" et les informe automatiquement de tout changement d'état.
- **Avantages** : Favorise un couplage faible entre les objets.
- **Limites** : Si mal géré, peut entraîner des fuites de mémoire ou des notifications en cascade imprévues.

2.3 Étude comparative des technologies Back-end

En entreprise, le choix d'une technologie se fait en comparant des indicateurs clés de performance (KPI). Nous avons sélectionné trois langages et évalué leur pertinence pour notre API.

Technologie	Courbe d'apprentissage	Écosystème (libs, frameworks)	Productivité
Python (ex: FastAPI)	Facile	Très riche (Data, API)	Très élevée
Node.js (ex: Express)	Moyenne	Très riche (NPM)	Élevée
Java (ex: Spring)	Difficile	Mature et robuste	Moyenne

Analyse par KPI (Key Performance Indicators)

Nous avons défini 4 indicateurs pour valider notre choix en contexte "entreprise" :

- **Productivité (Vitesse de développement)** : Capacité à livrer une fonctionnalité rapidement.
- **Courbe d'apprentissage** : Facilité pour un nouvel ingénieur de rejoindre le projet.
- **Performance (Latence)** : Rapidité de réponse aux requêtes des utilisateurs.
- **Écosystème** : Disponibilité de bibliothèques tierces (recommandation, accès DB).

2.4 Étude comparative des technologies Front-end

De la même manière, nous avons comparé plusieurs options pour le développement de l'interface utilisateur.

Technologie	Niveau de complexité	Maintenabilité	Cas d'usage typiques	Performance
HTML/CSS/JS pur	Faible	Faible sur de gros projets	Projets très simples, apprentissage	Maximale (Pas de surcouche)
React	Moyenne	Très bonne	Applications web modernes (SPA), standard entreprise	Très bonne (Virtual DOM)
Vue	Faible à moyen	Bonne	Bon compromis simplicité/puissance	Très bonne

2.5 Concepts fondamentaux du Web et de l'API

Les Environnements Virtuels

- Un environnement virtuel est un espace isolé sur l'ordinateur dédié à un projet spécifique. Pourquoi l'utiliser ? Il permet d'installer des bibliothèques (comme FastAPI) sans polluer le système global. Chaque projet peut avoir ses propres versions d'outils.
- **Problème résolu en entreprise :** Il élimine le problème du "Ça marche sur mon ordinateur, mais pas sur le serveur". En figeant les versions dans un environnement virtuel, on s'assure que tous les développeurs travaillent exactement avec les mêmes outils.

Les Formats de Données

Le JSON (JavaScript Object Notation) est un format de données textuel, léger et structuré. Bien qu'il soit dérivé de la syntaxe des objets JavaScript, il est totalement indépendant du langage de programmation. Il repose sur deux structures principales :

- **Des paires Clé : Valeur** (ex: "title": "Inception")

- Des Listes ordonnées de valeurs (Tableaux)

Le JSON s'est imposé comme le standard industriel pour plusieurs raisons :

- **Interopérabilité** : Il est lisible par quasiment tous les langages de programmation (Python, JS, Java, etc.).
- **Légèreté** : Sa structure est très concise par rapport à d'autres formats, ce qui réduit la bande passante nécessaire et accélère les transferts réseau.
- **Facilité d'utilisation** : Il est "human-readable" (lisible par l'homme) et très facile à transformer en objets manipulables dans le code (parsing).
- **Support natif du navigateur** : Comme le Web repose sur JavaScript, le format JSON est traité de manière optimale côté Front-end.

Comparaison avec d'autres formats

Il existe d'autres formats de données utilisés selon les besoins spécifiques du projet :

- **XML (eXtensible Markup Language)** :
 - Description : Format utilisant des balises personnalisées (similaire au HTML).
 - Cas d'usage : Très utilisé dans les anciens systèmes d'échange de données (SOAP) ou pour les fichiers de configuration complexes (ex: fichiers de projet Java Maven).
- **YAML (YAML Ain't Markup Language)** :
 - Description : Format basé sur l'indentation (les espaces), privilégiant la lisibilité humaine.
 - Cas d'usage : C'est le standard pour la configuration d'outils DevOps tels que Docker et Kubernetes, que nous utiliserons plus tard dans ce projet.
- **CSV (Comma-Separated Values)** :
 - Description : Format très simple où chaque ligne représente un enregistrement et chaque donnée est séparée par une virgule.
 - **Cas d'usage** : Idéal pour l'exportation de bases de données vers des tableurs comme Microsoft Excel.

API

Une API est une interface qui permet à deux logiciels de communiquer entre eux. Dans notre cas, il s'agit d'une API Web (HTTP).

- **Rôle** : Elle sert de "contrat" entre le Client (Front-end) et le Serveur (Back-end). Le serveur expose des points d'accès précis et le client sait exactement comment les appeler.
- **Endpoints (Routes)** : Ce sont les URL spécifiques disponibles (ex: /movies). Chaque endpoint correspond à une action précise (récupérer des films, en ajouter un, etc.).
- **Format de réponse** : L'API renvoie des données structurées (généralement du JSON) et non une page web visuelle (HTML).

API vs Web Scraping

Pour récupérer des données (comme notre catalogue de films), deux approches existent :

- **L'utilisation d'une API (Ex: TMDB)** : C'est la voie officielle et recommandée. Elle s'accompagne d'une documentation technique claire, de conditions d'utilisation définies et gère des concepts de sécurité comme les clés d'API (Tokens) et les limitations de requêtes (rate limiting).
- **Le Web Scraping** : Cette méthode consiste à extraire les données directement depuis le code HTML des pages web. Bien que puissante, elle pose des questions éthiques et légales ; il faut impérativement respecter les conditions d'utilisation du site cible et consulter son fichier robots.txt pour vérifier ce qui est autorisé.

2.6 Communication front-end / back-end

CORS

Le CORS est une sécurité intégrée aux navigateurs web (Chrome, Firefox, etc.). Par défaut, pour des raisons de sécurité, un navigateur interdit à un site web (le "Front") de faire des requêtes vers un serveur (le "Back") si ces deux-là n'ont pas la même origine. Qu'est-ce qu'une "Origine" ?

C'est la combinaison de trois éléments :

- Le Protocole (http vs https)
- Le Domaine (localhost vs mon-api.com)
- Le Port (5173 vs 8000) Si l'un de ces trois éléments change, c'est une "Cross-Origin" (origine différente).

Le navigateur bloque l'appel pour te protéger. Imagine que tu sois connecté à ton compte bancaire (banque.com) et que, sur un autre onglet, tu visites un site malveillant (pirate.com). Sans le CORS, le site pirate.com pourrait envoyer une requête à banque.com en utilisant ta session (tes cookies) pour vider ton compte. Le navigateur dit donc : "Je bloque tout, sauf si le serveur m'autorise explicitement à laisser passer ce site spécifique."

Pour régler ça, on ne modifie pas le Front, on configure le Back. On dit au serveur : "Ok, j'accepte que le site situé à l'adresse `http://localhost:5173` me parle."

2.7 Bonnes pratiques avancées (API réelle)

Dans le cadre de l'intégration d'une API externe comme TMDB, il est crucial de prendre en compte les contraintes du monde réel. Voici une synthèse des concepts clés, accompagnée d'une analyse de mon implémentation et des évolutions possibles pour un environnement de production.

Synthèse

- **Rate limiting et stratégies** : Les API publiques limitent le nombre de requêtes pour protéger leurs serveurs (renvoyant souvent le code HTTP 429 Too Many Requests). Pour gérer cela sans faire planter l'application cliente, on utilise des stratégies de Retry (réessayer la requête) couplées à un algorithme d'Exponential Backoff (augmenter progressivement le temps d'attente entre chaque essai pour ne pas saturer le réseau) (Source : Documentation Google Cloud API Design).
- **Cache (local / serveur)** : Le cache permet de stocker temporairement des données fréquemment demandées. Un cache serveur (ex: Redis) évite de refaire des appels à l'API tierce, tandis qu'un cache local (ex: LocalStorage du navigateur web) évite au client de refaire des requêtes au serveur. (Source : MDN Web Docs - HTTP Caching).
- **Pagination** : Transférer de grandes quantités de données dégrade les performances. La pagination (offset/limit ou par numéro de page) divise les réponses en sous-ensembles gérables. (Source : TMDB API Documentation).
- **Sécurité (Gestion des secrets)** : Les clés d'API ne doivent jamais être exposées dans le code front-end (HTML/JS), car elles sont visibles par tous les utilisateurs. Le back-end agit comme un Proxy sécurisé, utilisant des variables d'environnement (.env) non incluses

dans les systèmes de versioning comme Git. (Source : OWASP Top 10 - Cryptographic Failures).

- **Logs et observabilité** : L'observabilité permet de surveiller la santé de l'application. Les logs (journaux d'événements) enregistrent les erreurs, les requêtes entrantes et les temps de réponse, ce qui est vital pour le débogage asynchrone. (Source : The Twelve-Factor App - Logs).

2.8 Automatisation : Théorie du CI/CD

L'approche CI/CD (Intégration Continue et Déploiement Continu) est fondamentale pour garantir la qualité du code :

- **L'Intégration Continue (CI)** : C'est l'automatisation des tests et la validation du code à chaque nouvelle modification poussée sur le dépôt, permettant une détection rapide des erreurs.
- **Le Déploiement Continu (CD)** : C'est l'automatisation de la mise en production (ou déploiement) de l'application une fois que les tests de l'intégration continue sont validés.
- **Pipelines et Runners** : Un pipeline est une séquence de tâches (jobs regroupés en stages) exécutées automatiquement. Ces tâches sont réalisées par des Runners, qui sont des agents d'exécution (locaux ou distants) chargés de faire tourner les scripts.

2.9 Docker

Architecture fondamentale de Docker

Docker permet de standardiser l'environnement d'exécution de notre application. Il repose sur trois concepts théoriques majeurs :

- **L'Image** : C'est un modèle immuable construit en couches (layers) qui contient le code, les bibliothèques et l'environnement nécessaires.
- **Le Conteneur** : C'est un processus isolé (utilisant les namespaces et cgroups de Linux) en cours d'exécution, créé à partir d'une image.
- **Différence avec une Machine Virtuelle (VM)** : Contrairement à une VM qui embarque un système d'exploitation complet (OS) et nécessite un hyperviseur lourd, un conteneur Docker partage le noyau de l'OS hôte, ce qui le rend beaucoup plus léger et rapide à démarrer.

Les Réseaux Docker (Networking)

- **Le mode Bridge** : C'est un réseau virtuel NAT où il est nécessaire de publier les ports pour y accéder depuis l'hôte.
- **Le mode Host** : Ce mode partage directement le réseau de la machine hôte, ce qui implique une isolation moindre.
- **Les réseaux personnalisés (User-defined)** : Il faut mentionner que créer un réseau personnalisé permet aux conteneurs de communiquer entre eux via leurs noms grâce à un DNS interne fourni par Docker.

Optimisation et gestion du Cache (Dockerfile)

- **Le mécanisme de cache** : Expliquez l'importance de l'ordre des instructions dans un Dockerfile. Par exemple, copier le fichier requirements.txt avant le reste du code permet d'utiliser le cache de Docker et d'éviter de réinstaller toutes les dépendances à chaque modification du code source.
- **Le poids des images** : Mentionnez l'intérêt d'utiliser des images de base allégées (comme python:slim) pour réduire la taille finale de l'image.

2.10 Orchestration de conteneurs avec Kubernetes

- **Les Pods** : C'est l'unité d'exécution de base de Kubernetes, qui s'exécute sur un nœud (node) et qui embarque un ou plusieurs conteneurs à partir d'images Docker. Il est important de préciser que, par défaut, un Pod n'est pas accessible depuis l'extérieur du cluster.
- **Les Deployments** : C'est la ressource qui permet de gérer plusieurs Pods en même temps. Vous pouvez expliquer qu'il répond à deux enjeux majeurs :
 - La mise à l'échelle (Scaling) : Il permet d'augmenter le nombre de répliques (replicas) de votre application pour s'adapter à la demande (scaling horizontal).
 - La résilience : Si un Pod subit une panne ou est supprimé, le Deployment se charge automatiquement d'en créer un nouveau pour maintenir l'application en ligne.

- **Les Services :** Puisqu'un Pod est isolé, le Service est le composant théorique qui permet d'exposer l'application vers l'extérieur (par exemple via un NodePort) en lui attribuant un port d'accès.
- **La Gestion de la Configuration :**
 - **Le format YAML :** Il faut expliquer que Kubernetes fonctionne de manière déclarative via des fichiers YAML, qui décrivent l'état souhaité de l'infrastructure.
 - **Les ConfigMaps :** Ce sont des objets utilisés pour stocker la configuration (comme des variables d'environnement, par exemple ENV=dev), ce qui permet de bien séparer le code de l'application de sa configuration.

3 Partie Pratique

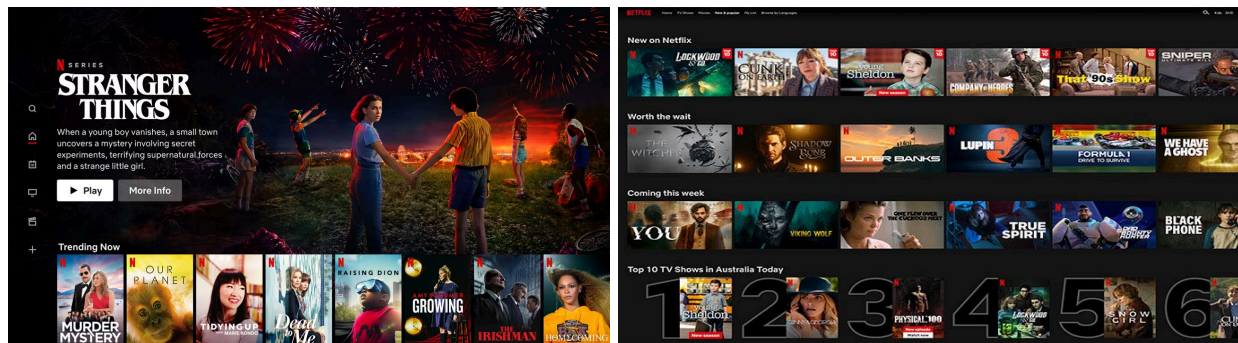
3.1 Contexte général - Netplix

3.1.1 Idée du projet

Notre application web complète s'appelle **Netplix**. Il s'agit d'un catalogue interactif contenant plus de 25 000 films, qui offre aux utilisateurs la possibilité de parcourir, rechercher et filtrer des œuvres cinématographiques. L'application met à disposition des fonctionnalités de base telles que l'affichage de fiches détaillées, ainsi que des fonctionnalités avancées incluant un système de sauvegarde de favoris, des recommandations de films similaires basées sur les genres, et une recherche tolérante aux fautes de frappe.

3.1.2 Inspiration

L'interface et l'expérience utilisateur de Netplix sont directement inspirées de la célèbre plateforme de streaming **Netflix**. Nous nous sommes inspirés de l'organisation du catalogue en rangées horizontales défilantes ("Netflix-style").



3.1.3 Architecture choisie

Nous avons choisi l'architecture client-serveur en couches.

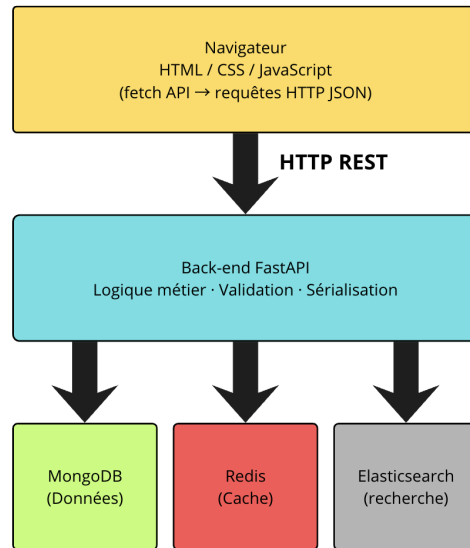


Figure 1 : Schéma client serveur en couches de Neplix

Car client serveur en couche est naturellement adaptée à Docker : chaque composant (front-end, back-end, base de données) devient un conteneur isolé, elle facilite le déploiement Kubernetes via des services distincts, et respecte le principe de séparation des responsabilités : le front-end gère l'affichage, le back-end gère la logique, la base de données gère la persistance, et le plus important, elle est plus maîtrisable pour une équipe de deux développeurs.

3.1.4 Design patterns choisie

Le développement du projet a été structuré autour de trois grands design patterns :

- **Pattern MVC (Model-View-Controller):**
 - **Model :** la collection MongoDB shows et les fonctions de mapping (`clean_show`, `import_csv`)
 - **View :** les fichiers `index.html`, `style.css` et les fonctions de rendu JS (`buildCard`, `openModal`)
 - **Controller :** les routes FastAPI (`get_shows`, `search_shows`, `get_similar`, etc.)
- **Pattern Repository**

Le fichier `database.py` joue le rôle de couche d'abstraction entre la logique métier et le stockage. La variable `shows_collection` encapsule l'accès à MongoDB. Si nous décidons de migrer vers PostgreSQL, seul `database.py` serait à modifier.

- **Pattern Singleton**

La connexion MongoDB (MongoClient) est instanciée une seule fois dans database.py et importée dans main.py. Python garantit qu'un module importé n'est évalué qu'une seule fois.

3.1.5 Explication de la stack technologique choisie

La stack technologique a été sélectionnée pour répondre aux exigences de performances modernes tout en validant les objectifs pédagogiques liés au cloud.

- **Front-end** : nous avons utilisé le HTML, CSS et JavaScript Vanilla afin de garantir des performances maximales d'exécution sans imposer de dépendances de build lourdes.
- **Back-end** : comme dans les TMEs, le choix de Python 3.11 couplé au framework FastAPI, favorisant une productivité élevée, un typage natif robuste et la génération automatique d'une documentation Swagger.
- **Données et Cache** : nous avons eu l'idée de faire NoSQL avec MongoDB pour sa grande flexibilité, soutenu par Redis pour la mise en cache ultra-rapide des requêtes API, et par Elasticsearch pour le moteur de recherche "full-text" tolérant aux fautes.
- **Conteneurisation** : Docker et Docker Compose. Chaque composant (front-end, back-end, bases de données) dispose de son propre conteneur immuable, pour que l'application fonctionne de manière identique sur n'importe quelle machine.
- **Orchestration** : Kubernetes. K8s prend le relais de Docker Compose pour gérer le cycle de vie des conteneurs, assurer la résilience de l'application (relance automatique en cas de panne) et gérer le routage du trafic réseau via un Ingress.

3.1.6 Organisation du travail

Tasks / Nom	Mohamed Yassine Agourram	Kun Li
Front-end (Interface Utilisateur)		
Back-end (API RESTful)		
Base de données principale		
Système de Cache en mémoire		
Moteur de recherche avancé		
Conteneurisation		
Orchestration		
Intégration et Déploiement Continu (CI/CD)		

3.2 Source des données

3.2.1 Choix du dataset

Pour alimenter notre catalogue, nous avons fait le choix architectural d'utiliser un dataset statique plutôt que de consommer une API externe en temps réel (comme l'API TMDb étudiée lors du TME 3).

Une sélection basée sur la qualification des données (Data Profiling)

Ce choix de la source de données ne s'est pas fait de manière arbitraire, mais est le fruit d'une démarche itérative. Avant d'arrêter notre choix sur notre dataset actuel, nous avons évalué et qualifié plusieurs bases de données populaires sur la plateforme Kaggle, notamment :

1. Netflix Movies and TV Shows (Dataset historique Netflix)
2. TMDb Top 10K Movies
3. IMDb Top 5000 Movies

Ces sources ont été écartées à l'issue de notre phase d'analyse pour deux raisons majeures :

- **Le volume de données :** Certains datasets (5 000 à 10 000 entrées) présentaient un volume insuffisant pour démontrer de manière convaincante les capacités de notre moteur de recherche Elasticsearch ou de notre pagination.

- **La complétude des données (Assets visuels) :** D'autres bases de données, bien que volumineuses, souffraient d'une carence critique en métadonnées, et plus particulièrement de l'absence d'URLs pointant vers les affiches de films.

L'enseignement de notre premier MVP (Minimum Viable Product)

Pour valider l'impact de ces données manquantes, nous avons développé une première version de notre application (MVP) en utilisant le dataset Netflix Shows. Si l'interface utilisateur (UI) et la logique de routage du back-end fonctionnaient parfaitement sur le plan technique, le résultat visuel était inexploitable. L'absence d'affiches brisait totalement l'expérience utilisateur (UX) requise pour une plateforme de streaming vidéo, l'interface se retrouvant remplie d'espaces vides ou dans notre cas avec des emojis.

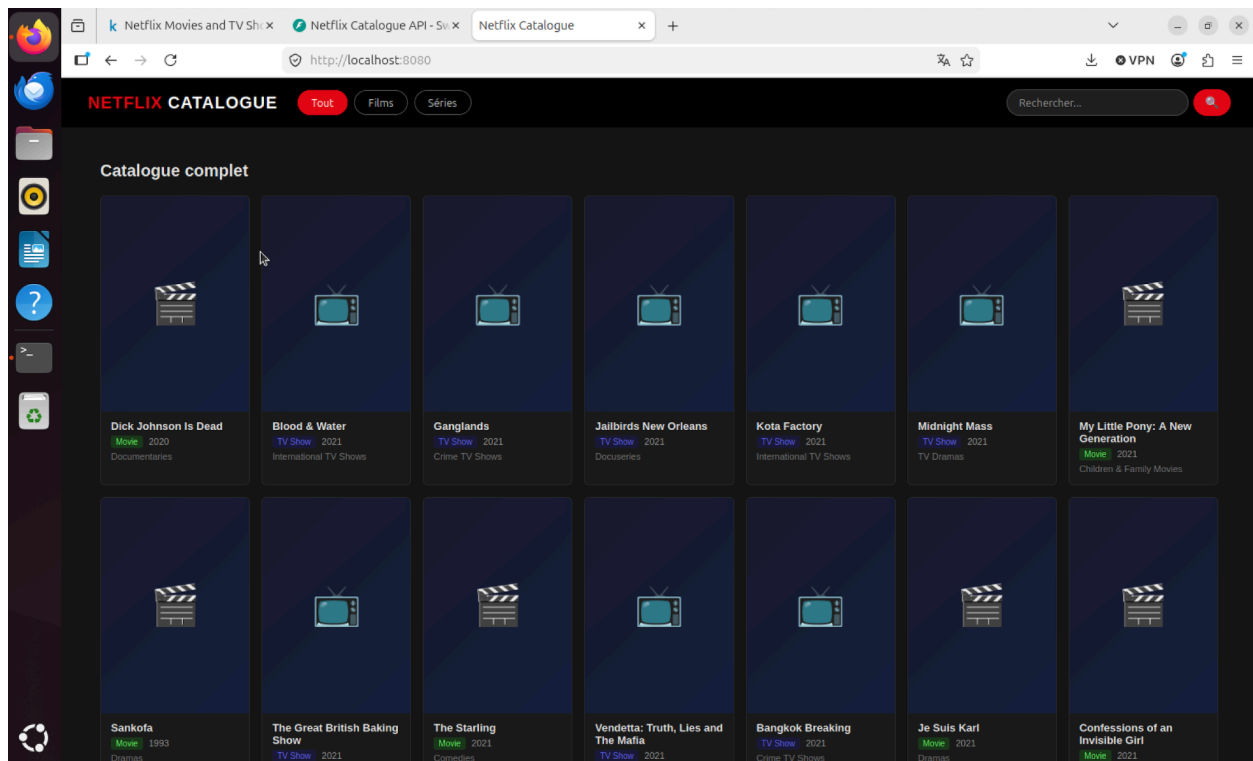


Figure : Interface du premier MVP utilisant un dataset incomplet, illustrant la dégradation de l'expérience utilisateur due à l'absence d'assets visuels.

C'est ce problème qui nous a poussés à pivoter vers notre dataset Kaggle actuel combinant les données TMDB (The Movie Database) et IMDb plutôt qu'une API externe. Ce dataset contient

25 000 films avec les champs suivants : title, overview (description), genres, vote_average (note sur 10), release_date, poster_path (chemin vers l'affiche), vote_count.

Le lien: <https://www.kaggle.com/datasets/ggtejas/tmdb-imdb-merged-movies-dataset>

Justification finale (Dataset vs API TMDB) :

Critère	Dataset Kaggle	API TMDB en direct
Volume de données	25 000 films disponibles immédiatement	Limité par les quotas (40 req/10s)
Indépendance	Aucune clé API requise	Clé exposable, quota épuisable
Performance	Chargement unique en base	Latence réseau à chaque requête
Disponibilité	100% offline	Dépend de l'infrastructure TMDB

3.2.1 Import et normalisation des données

Le script backend/import_data.py réalise le pipeline d'import :

Analyse des décisions techniques de l'implémentation :

- **Gestion de la qualité des données** : La bibliothèque pandas représente nativement les cellules vides par float('nan'). MongoDB ne gérant pas ce type de manière optimale, une conversion explicite vers le type None (qui devient null en JSON) est appliquée pour garantir l'intégrité du schéma.
- **Normalisation sémantique** : Les clés issues de TMDB sont transformées pour correspondre au contrat d'interface de notre API (overview devient description, vote_average devient rating).
- **Performances d'insertion** : L'utilisation d'une insertion par lots plutôt que 25 000 appels à insert_one réduit drastiquement les allers-retours réseau entre le serveur Python et le conteneur MongoDB, divisant le temps d'import par dix.
- **Création d'Index B-Tree** : La création d'index sur les champs fréquemment interrogés (title, listed_in, rating) est importante pour garantir des temps de réponse rapide même avec un volume de données croissant.

3.2.3 Import automatique au démarrage

Dans un environnement distribué géré par Kubernetes, l'application doit être capable de s'initialiser de manière autonome. C'est pourquoi nous avons couplé l'import des données au cycle de vie du framework FastAPI via l'événement startup.

```
@app.on_event("startup")
def startup_event():
    if shows_collection.count_documents({}) == 0:
        import_csv()
        _es_ensure_index()
        _es_index_all()
```

En vérifiant d'abord si la collection est vide (`count_documents({}) == 0`), nous nous assurons que le redémarrage d'un pod ne provoquera pas la duplication des 25 000 films. L'application est ainsi responsable de son propre état initial et synchronise automatiquement la base de données principale (MongoDB) avec le moteur de recherche secondaire (Elasticsearch) dès son lancement.

3.3 Back-end : FastAPI et MongoDB

3.3.1 Connexion au services - database.py

Ce fichier est le point d'entrée de toutes les connexions aux services externes, impliquant une stratégie de dégradation gracieuse.

Points techniques importants:

- **Dégradation gracieuse :** Les blocs try/except autour des connexions Redis et Elasticsearch garantissent que si ces services tombent, l'API ne crashe pas. Elle basculera automatiquement sur un mode dégradé, utilisant uniquement MongoDB pour les lectures et la recherche.
- **Méthode de configuration :** L'utilisation stricte de `os.getenv()` permet de séparer complètement le code de sa configuration. L'URL de connexion n'est jamais écrite en dur. L'application utilise localhost ; en production, Kubernetes injecte les adresses internes du cluster (ex: `mongodb://mongo-service:27017`) via les variables d'environnement du ConfigMap.
- **Principe du timeout :** Le paramètre `socket_connect_timeout=2` pour Redis et `request_timeout=5` pour Elasticsearch sont importants. Sans eux, si le réseau subit une

latence, le thread Python pourrait rester bloqué indéfiniment en attendant une réponse, provoquant l'écroulement de l'API. Ici, l'application abandonne au bout de 2 secondes et continue son exécution.

- **Optimisation de la sérialisation (decode_responses=True) :** Par défaut, le client Redis retourne des données sous forme d'octets bruts (bytes). Redis stocke nativement des bytes, alors cette option fait la conversion automatique en strings Python.

3.3.2 API principale - main.py

Configuration de l'application FastAPI

CORS (Cross-Origin Resource Sharing) : une sécurité du navigateur bloque les requêtes JavaScript vers un domaine ou un port différent. Sans cette configuration, le front-end sur localhost:8080 (qui devient après http://netflix.local/) ne pourrait pas appeler le back-end sur localhost:8000. `allow_origins=["*"]` autorise toutes les origines

Le système de cache Redis

```
def make_key(*args) -> str:
    raw = ":".join(str(a) for a in args)
    return "netplixe:" + hashlib.md5(raw.encode()).hexdigest()

def cache_get(key: str):
    if not redis_client:
        return None
    val = redis_client.get(key)
    return json.loads(val) if val else None

def cache_set(key: str, data, ttl: int = 300):
    if not redis_client:
        return
    redis_client.setex(key, ttl, json.dumps(data, default=str))
```

La fonction `make_key` génère une clé unique pour chaque combinaison de paramètres en calculant un hash. Par exemple, la requête `GET /shows?limit=20&sort_by=rating` génère toujours la même clé hexadécimale. `setex` (SET with EXpiration) stocke la donnée avec une durée de vie automatique : après `ttl` secondes, Redis supprime la clé et la prochaine requête régénèrera la réponse, évitant ainsi la saturation du cache et garantissant la fraîcheur des données.

Endpoint /health - Sonde de disponibilité

Cet endpoint est utilisé par Kubernetes (K8s) qui l'appelle toutes les 10 secondes pour savoir si le pod est prêt à recevoir du trafic. Si la réponse n'est pas HTTP 200, K8s redirige le trafic vers d'autres pods.

Endpoint /shows - Catalogue paginé avec filtres

L'utilisation de la classe Query de FastAPI permet d'appliquer le principe de Validation de Données (Data Validation) directement à la définition de la route. Par exemple, Query(ge=1, le=200) impose des bornes strictes. Si un client tente une requête mal formée (ex: limit=999), le framework intercepte l'appel et retourne automatiquement une erreur HTTP 422 Unprocessable Entity sans que nous ayons à écrire de code de validation manuellement.

La réponse inclut les métadonnées de pagination :

```
result = {
    "items": items,
    "total": total,
    "page": (skip // limit) + 1,
    "pages": math.ceil(total / limit),
    "from_cache": False,
}
cache_set(cache_key, result) # Mise en cache 5 minutes
```

Le front-end utilise total, page et pages pour construire les boutons de navigation.

Endpoint /shows/search - Recherche simple

\$or combine plusieurs conditions : le film correspond si le terme de recherche apparaît dans le titre, la description ou les genres. .sort("rating", -1) trie les résultats par note décroissante les films les mieux notés apparaissent en premier.

Endpoint /shows/favorites Favoris

Les favoris sont stockés côté client dans le localStorage du navigateur (uniquement leurs IDs). Quand l'utilisateur clique sur "Mes Favoris", le front-end envoie la liste des IDs au back-end via une requête POST, et le back-end retourne les documents complets. Ce design présente plusieurs avantages : les données des films restent toujours à jour (on lit en base à chaque fois), et le back-end n'a pas besoin de gérer des sessions utilisateurs. ObjectId.is_valid(id_) protège contre les IDs malformés qui provoqueraient une erreur MongoDB.

Endpoint /shows/{id}/similar - Films similaires

Algorithme de recommandation :

1. Extraire les genres du film demandé (ex : ["Action", "Crime", "Drama"])
2. Construire une regex MongoDB : Action|Crime|Drama
3. Chercher les films ayant au moins un genre en commun, avec une note ≥ 6.0
4. Récupérer jusqu'à 500 candidats (50× la limite souhaitée) pour compenser les doublons du dataset
5. Dédupliquer par clé titre+année en Python
6. Retourner les limit premiers films uniques

Le multiplicateur ×50 est nécessaire car le dataset contient des doublons. Sans lui, nous risquerions de renvoyer plusieurs fois le même film qui était le problème qui nous a poussé à faire cette solution.

3.3.3 Elasticsearch - Recherche avancée

Création et remplissage de l'index

```
def _es_ensure_index():
    if not es_client.indices.exists(index="shows"):
        es_client.indices.create(
            index="shows",
            mappings={
                "properties": {
                    "title": {"type": "text", "analyzer": "standard"},
                    "description": {"type": "text", "analyzer": "standard"},
                    "listed_in": {"type": "text"},
                    "rating": {"type": "float"},
                    "release_year": {"type": "keyword"},
                }
            }
        )

def _es_index_all():
    from elasticsearch.helpers import bulk
    def _generate():
        for doc in shows_collection.find({}):
            mid = str(doc['_id'])
            yield {
                "_index": "shows",
                "_id": mid,
                "_source": { "mongo_id": mid, "title": doc.get("title"), ... }
            }
    bulk(es_client, _generate(), chunk_size=500)
```

bulk indexe les documents par lots de 500, ce qui est bien plus efficace que 25 000 insertions individuelles.

Requête de recherche avancée

- **title³** : le titre a 3× plus de poids que la description. Un film dont le titre correspond exactement sera classé plus haut qu'un film qui ne mentionne le terme que dans sa description.
- **fuzziness: "AUTO"** : tolérance automatique aux fautes de frappe. "Avenjers" trouve "Avengers". Elasticsearch calcule la distance d'édition de Levenshtein pour décider si deux mots sont "proches".

Après la recherche Elasticsearch, nous récupérons les données complètes depuis MongoDB :

```
mongo_ids = [hit["_id"] for hit in hits]
docs_map = {str(d["_id"]): d for d in shows_collection.find(
    {"_id": {"$in": [ObjectId(mid) for mid in mongo_ids]}}
)}
items = [clean_show(docs_map[mid]) for mid in mongo_ids if mid in docs_map]
```

ES stocke les champs indexés pour la recherche (titre, genres). MongoDB stocke les données complètes (affiche, description longue). ES donne les IDs dans le bon ordre de pertinence ; MongoDB donne les documents complets.

3.4 Front-end HTML/CSS/JavaScript

L'interface utilisateur a été développée sans l'usage de frameworks lourds, en s'appuyant sur les standards du Web (HTML5, CSS3, et JavaScript "Vanilla") afin de maximiser les performances et de garantir une maîtrise totale du code.

3.4.1 Structure HTML

L'architecture de la page (index.html) est conçue pour être modulaire et facilement manipulable par JavaScript.

Navigation et transmission de données

La barre de navigation utilise intensivement les attributs de données HTML5 (data-type, data-genre) pour transmettre des informations au JavaScript de manière propre, sans inclure de logique directement dans le balisage.

Composants visuels clés

La section d'en-tête ("Hero") emploie un système de couches avec une image de fond (hero-backdrop) et un dégradé superposé (hero-overlay) pour reproduire fidèlement l'esthétique de Netflix. Les catalogues de films sont structurés dans des conteneurs flexibles (flex) permettant un défilement horizontal (overflow-x: auto).

Éléments interactifs

L'interface intègre un panneau de recherche avancée combinant de multiples critères, ainsi qu'une fenêtre modale composée de deux couches (un fond cliquable et le conteneur de contenu) qui se remplit dynamiquement à l'ouverture.

3.4.2 Design CSS

Le fichier style.css gère l'intégralité de la présentation visuelle et des animations.

Cohérence visuelle

L'utilisation de variables CSS (pseudo-classe :root) permet de centraliser le thème de l'application (comme le "Rouge Netflix", les couleurs de fond et de texte), facilitant ainsi sa maintenance.

Animations et fluidité

Des transitions fluides sont appliquées à la barre de navigation, qui passe de transparente à opaque lors du défilement de la page. Les cartes de films intègrent un effet de survol sophistiqué utilisant transform: scale(1.1) et une gestion précise du z-index pour faire ressortir l'élément survolé.

Adaptabilité

Le design est entièrement responsive. Des requêtes multimédias (media queries) ajustent dynamiquement l'interface (taille des polices, masquage de certains menus, redimensionnement des cartes) pour les écrans de tablettes et de téléphones mobiles.

3.4.3 Logique JavaScript

Le fichier script.js orchestre les interactions, la communication avec l'API et la gestion de l'état de l'application.

Communication réseau optimisée

Les appels à l'API sont réalisés de manière asynchrone via la fonction fetch. L'objet URLSearchParams est utilisé pour construire dynamiquement les requêtes, en filtrant automatiquement les paramètres vides ou non définis. Au chargement initial, les différentes rangées du catalogue sont appelées en parallèle grâce à Promise.all, réduisant drastiquement le temps de rendu global.

Gestion des données locales

Le système de favoris est géré côté client via l'API localStorage, permettant de sauvegarder les identifiants des films préférés du navigateur de l'utilisateur afin qu'ils persistent même après la fermeture de l'onglet. Une fonction de déduplication utilisant l'objet Set est également implémentée pour filtrer les doublons potentiels du dataset en se basant sur le titre et l'année de sortie.

Sécurité et performances

Lors de la construction dynamique des cartes HTML, une fonction d'échappement (esc()) convertit les caractères spéciaux pour prévenir toute vulnérabilité aux failles XSS (Cross-Site Scripting). De plus, l'attribut loading="lazy" est appliqué aux affiches de films pour que le navigateur ne télécharge les images que lorsqu'elles entrent dans le champ de vision de l'utilisateur, économisant ainsi une quantité significative de bande passante.

Pagination récursive

La navigation entre les pages de résultats repose sur un algorithme récursif utilisant des fonctions de rappel (callbacks). Ce système permet de recharger uniquement les données nécessaires et de mettre à jour dynamiquement les boutons de pagination sans jamais recharger la page complète.

3.5 Communication Front-end/Back-end

3.5.1 Le protocole REST

Notre API respecte les conventions RESTful :

Méthode	Route	Action
GET	/shows	Récupérer le catalogue paginé
GET	/shows/search?q=...	Recherche simple
GET	/shows/search/advanced	Recherche multi-critères
GET	/shows/{id}	Détail d'un film
GET	/shows/{id}/similar	Films similaires
POST	/shows/favorites	Charger les films favoris
GET	/shows/stats	Statistiques du catalogue
GET	/health	Sonde de disponibilité

3.5.2 Format de réponse uniforme

Toutes les réponses de liste partagent le même format JSON :

```
{
  "items":    [...],
  "total":    25000,
  "page":     2,
  "pages":    1250,
  "limit":    20,
  "skip":     20,
  "from_cache": true,
  "engine":   "elasticsearch"
}
```

Le champ `from_cache` indique si la réponse provient de Redis utile pour le monitoring et affiché dans la barre de statistiques de recherche. Le champ `engine` indique si Elasticsearch ou MongoDB a servi la requête.

3.5.3 Gestion des erreurs HTTP

```
@app.get("/shows/{show_id}")
def get_show(show_id: str):
    try:
        doc = shows_collection.find_one({"_id": ObjectId(show_id)})
    except Exception:
        raise HTTPException(status_code=400, detail="ID invalide")
    if not doc:
        raise HTTPException(status_code=404, detail="Show introuvable")
    return clean_show(doc)
```

- **400 Bad Request** : l'ID fourni n'est pas un ObjectId MongoDB valide
- **404 Not Found** : l'ID est valide mais aucun document ne correspond
- **422 Unprocessable Entity** : paramètres invalides (automatiquement géré par FastAPI via Query)

3.6 Conteneurisation avec Docker

Application Netplix s'exécute de manière identique quel que soit l'environnement (développement local ou serveur de production), l'ensemble du projet a été conteneurisé à l'aide de l'écosystème Docker.

3.6.1 Bonne pratique : .dockerignore

L'utilisation d'un fichier .dockerignore est une bonne pratique indispensable appliquée à nos environnements. On peut exclure les fichiers locaux inutiles (comme les environnements virtuels Python, les dossiers node_modules ou .git) lors de la copie du contexte vers le moteur Docker. Cela optimise grandement le temps de construction (build) et réduit la taille finale des images de l'application.

3.6.2 Dockerfile du back-end

Le back-end utilise l'image de base python:3.11-slim, qui supprime les outils de développement non essentiels pour réduire la taille de l'image de 900 Mo à environ 150 Mo. Une optimisation majeure du cache Docker a été mise en place : le fichier requirements.txt est copié et les dépendances sont installées avant le reste du code source. Ainsi, Docker réutilise cette couche en cache si seul le code métier est modifié, ce qui fait gagner de précieuses minutes à chaque compilation.

3.6.3 Dockerfile du front-end

Le front-end s'appuie sur l'image `nginx:alpine`, on l'utilise car il est très performant et très léger, pesant à peine 40 Mo. Il se charge exclusivement de servir les fichiers statiques de l'application (HTML, CSS, JS), une tâche pour laquelle il est beaucoup plus adapté que le framework FastAPI.

3.6.4 Docker compose

Le fichier `docker-compose.yml` orchestre les différents services (MongoDB, Redis, Elasticsearch, API et interface).

- **Healthchecks et dépendances** : Il intègre des sondes de disponibilité (healthchecks) pour s'assurer de l'état des bases de données avant de lancer le back-end, grâce à la directive `depends_on: condition: service_healthy`.
- **Résolution DNS interne** : Les services communiquent entre eux via le nom des conteneurs (ex: l'URL `mongodb://mongo:27017` pointe directement vers le conteneur MongoDB).
- **Persistance** : Des volumes nommés (`mongo_data`, `es_data`) sont configurés pour garantir la persistance des données entre les redémarrages de l'environnement.

3.6.5 Réseau Docker

L'ensemble des services de l'application partage un réseau virtuel privé configuré en mode bridge. Seuls les ports explicitement déclarés (comme le port 8000 pour le back-end et 8080 pour le front-end) sont publiés et accessibles depuis l'extérieur du réseau Docker.

3.6.6 Problèmes rencontrée

Après la modification du code et la reconstruction de l'image, l'orchestrateur continuait d'utiliser l'ancienne version. La cause provenait de la politique d'image `imagePullPolicy: IfNotPresent` couplée au tag `latest`, qui indiquait au système d'utiliser l'image locale existante sans détecter les changements de code.

Nous avons fini à forcer l'utilisation exclusive de l'image locale nouvellement construite via `imagePullPolicy: Never`, et à forcer le redémarrage des déploiements pour appliquer la nouvelle image.

3.7 Orchestration avec Kubernetes

Pour passer d'une application conteneurisée à un système distribué résilient, nous avons choisi Kubernetes comme orchestrateur. Contrairement à une exécution locale via Docker Compose, Kubernetes fonctionne selon un modèle déclaratif : nous décrivons l'état souhaité (le `Desired State`) via des manifestes YAML, et la boucle de réconciliation de l'orchestrateur s'assure en permanence que la réalité correspond à cet état.

Concept	Rôle
Namespace	Espace de noms isolé pour regrouper les ressources
Pod	Unité d'exécution (un ou plusieurs conteneurs)
Deployment	Décrit l'état désiré (image, replicas, variables d'env)
Service	Adresse réseau stable pour accéder aux pods
ConfigMap	Variables de configuration non-secrètes
Secret	Variables sensibles (mots de passe, clés API)
Ingress	Routeur HTTP exposant l'application vers l'extérieur

3.7.1 Organisation en Namespace

Toutes nos ressources sont regroupées dans le namespace `netflix-app`. Cela isole le projet des ressources système de Kubernetes et facilite la gestion (`kubectl get all -n netflix-app`).

3.7.2 ConfigMap - Configuration centralisée

Le ConfigMap centralise la configuration. Les deployments référencent ce ConfigMap via `envFrom: configMapRef`. Si l'URL de MongoDB change, on modifie uniquement le ConfigMap.

En Kubernetes, les services se trouvent via le DNS interne de K8s mongo-service est automatiquement résolu vers l'IP du pod MongoDB.

3.7.3 Deployment du back-end

- **replicas: 1** : un seul pod back-end. Nous avons délibérément choisi une seule réplique pour éviter une race condition au démarrage : si deux pods démarrent simultanément et que la base est vide, les deux exécutent `import_csv()` en même temps, doublant les données. Un seul pod résout ce problème.
- **imagePullPolicy: Never** : indique à Kubernetes d'utiliser l'image locale (chargée dans Minikube via `minikube image load`) et de ne jamais essayer de la télécharger depuis Docker Hub.
- **readinessProbe** : K8s appelle `GET /health` toutes les 10 secondes. Tant que la probe ne répond pas HTTP 200, K8s ne route pas de trafic vers ce pod. `initialDelaySeconds: 15` laisse le temps au serveur FastAPI de démarrer et d'importer les données.
- **resources** : `requests` = ressources que Kubernetes garantit (le pod est schedulé sur un nœud avec au moins ces ressources disponibles) ; `limits` = maximum absolu (si dépassé, K8s termine le pod avec OOMKill pour tuer et redémarrer le conteneur, protégeant ainsi le reste du cluster.).

3.7.4 Deployment du front-end

Contrairement au Back-end, le front-end peut avoir 2 répliques car Nginx servant des fichiers statiques est entièrement stateless. Si un pod Nginx tombe, l'autre continue de servir l'application. Les ressources sont très faibles (32Mi RAM) car Nginx est extrêmement léger comparé au back-end.

3.7.5 L'Ingress - Routeur HTTP

L'Ingress est le point d'entrée unique de l'application. Toutes les requêtes arrivent sur `netflix.local` et sont routées selon le chemin. Si le client web requête `[http://netflix.local/api/shows](http://netflix.local/api/shows)`, l'Ingress intercepte l'appel, "découpe" le préfixe `/api`, et transmet la requête `/shows` pure au Back-end. Cette encapsulation

cache la structure interne de l'API à l'utilisateur final. Le trafic racine (/) est, quant à lui, acheminé vers le service Front-end (Nginx).

3.7.6 Services Redis et Elasticsearch déployés

Pour des raisons évidentes de cybersécurité, nos bases de données ne sont pas exposées sur Internet. Les services Redis et Elasticsearch utilisent le type ClusterIP accessibles uniquement depuis l'intérieur du cluster. Ils ne sont jamais exposés à l'extérieur, ce qui est une bonne pratique de sécurité. Seul le Back-end possède les privilèges nécessaires pour s'y connecter.

3.8 CI/CD et dépôt Git

Une approche DevOps a été adoptée avec la mise en place d'un pipeline d'Intégration Continue et de Déploiement Continu (CI/CD).

Avec Dépôt Git : <https://github.com/MeohamedYassineAgourram/netflix-catalogue>

3.8.1 Déroulement du pipeline

- **Tests du back-end** : Le pipeline configure un environnement virtuel Ubuntu, déploie un conteneur MongoDB temporaire, installe les dépendances Python, et démarre l'API. Une simple requête automatisée sur l'endpoint /health permet de valider que le serveur démarre et répond correctement (code 200).
- **Build des images Docker** : Si les tests du back-end sont validés, ce second travail prend le relais. Il vérifie la syntaxe des Dockerfile et construit les nouvelles images du front-end et du back-end, assurant que l'application est toujours dans un état déployable en production.

4 Conclusion

Cette Unité d'Enseignement représente une véritable opportunité d'immersion dans la culture et les pratiques DevOps. Elle nous a permis de traverser l'ensemble du cycle de vie d'une application, de sa conception initiale jusqu'à sa conteneurisation et son déploiement orchestré.

Dès le début du semestre, notre ambition dépassait la simple validation académique du module : nous visions la conception d'un produit logiciel complet, robuste et répondant aux standards de l'industrie professionnelle. Pour y parvenir, nous avons adopté une démarche proactive, alliant l'assimilation rigoureuse des concepts abordés en cours à une veille technologique approfondie en autodidactes.

Le choix de s'inspirer de Netflix s'est imposé naturellement. Au-delà de son interface utilisateur (UI) immersive, cette plateforme intègre des défis techniques (gestion de catalogue massif, moteur de recherche, mise en cache) parfaitement alignés avec les exigences fonctionnelles de notre projet.

Sur le plan méthodologique, nous avons privilégié une approche de développement itérative. La livraison initiale d'un premier modèle fonctionnel (Produit Minimum Viable - MVP) nous a servi de socle technique. Nous y avons ensuite greffé, étape par étape, les fonctionnalités avancées requises par les consignes. L'intégration de ces nouveaux concepts (Elasticsearch, Redis, LocalStorage) a nécessité un apprentissage en continu tout au long du développement afin d'en maîtriser les rouages et de les implémenter de manière optimale.

Finalement, le projet "Netplix" est l'aboutissement d'un véritable travail d'équipe. Il nous a permis de simuler avec succès la dynamique d'un environnement professionnel, de surmonter des défis techniques concrets, et d'acquérir des compétences solides et transversales qui nous seront précieuses pour notre future carrière d'ingénieurs.

5 Références

- Documentation officielle de FastAPI (fastapi.tiangolo.com)
- Microsoft API Design Guidelines
- Blog technique "Refactoring Guru"
- Documentation Google Cloud API Design
- MDN Web Docs - HTTP Caching
- TMDB API Documentation
- OWASP Top 10 - Cryptographic Failures
- The Twelve-Factor App - Logs